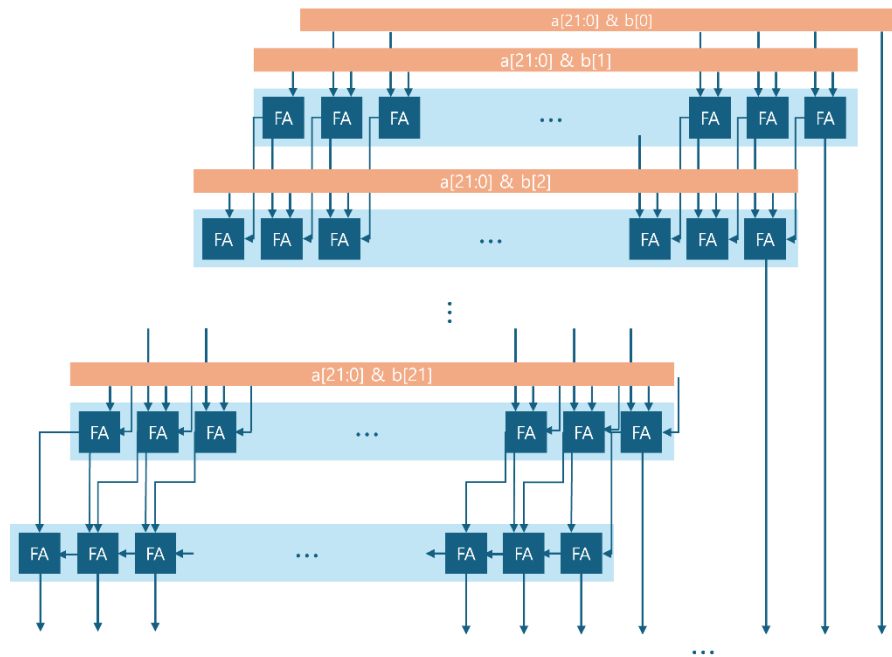


VLSI Practice 05 Report

2022171005 권민찬

1. 22-bit Carry Save Multiplier with RCA

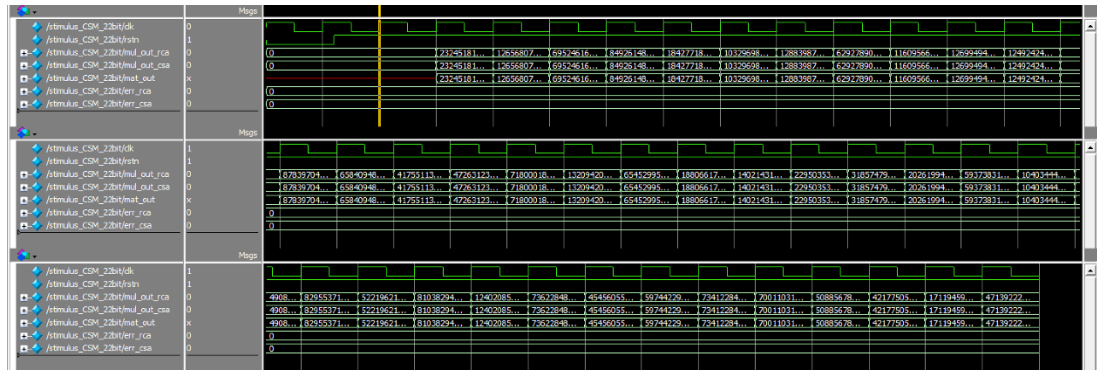
a. Architecture



Carry save multiplier은 곱셈기 내부의 수많은 가산기가 이전 행의 가산기의 carry out을 기다리면서 생기는 병목을 해소하기 위해 고안됐다. 이번 실험에서 합성할 CSM_RCA의 구조는 위의 그림과 같다. 총 21개의 22-bit adder가 존재하고, 마지막에는 22-bit RCA가 vector merging adder로 기능한다. 21개의 가산기는 carry가 ripple하는 형태가 아니라, 이전 행의 가산기에서 생성된 carry를 바로 받기 때문에, carry propagation을 기다릴 필요가 없게 된다.

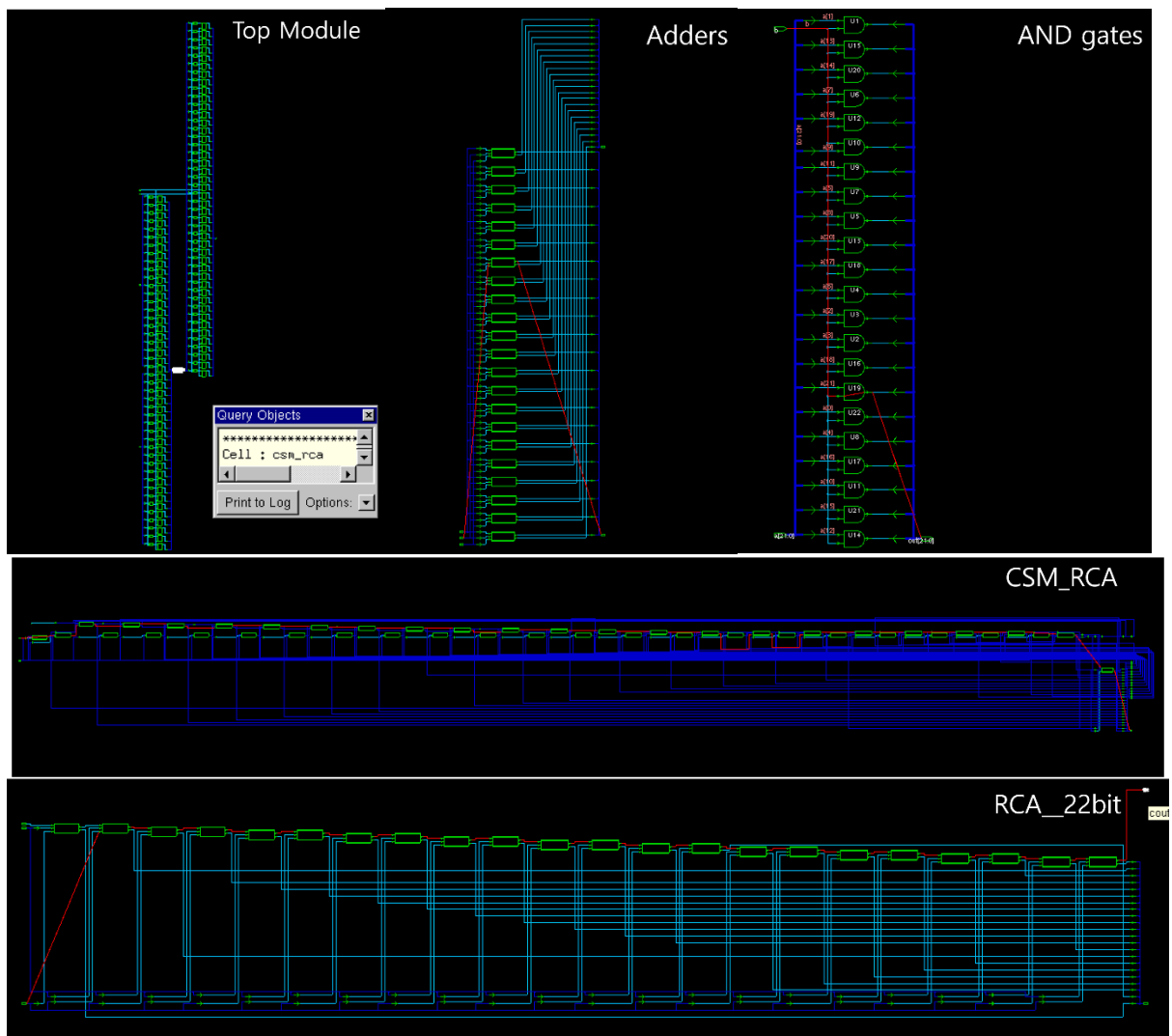
이번 설계에서는 HA를 쓰지 않고 FA만을 이용했다. 그림의 맨 왼쪽에 있는 FA들은 b핀을 0으로 연결해주어 HA를 구현했고, 맨 위의 FA들은 cin핀을 0으로 연결해주어 HA를 구현했다. 마지막으로 맨 아랫줄의 최우측 FA는 b포트를 0으로 연결해서 HA로 기능하게 만들었다. 제대로 된 HA를 이용해 설계한다면 면적을 최적화할 수 있을 것이다.

b. Pre-Synthesis Simulation



위 사진은 시뮬레이션 파형의 처음, 중간, 끝부분을 나타낸 것이다. 'mul_out_rca'가 곱셈 결과를 출력하고 있고, mat_out의 파형과 완전히 일치하고 있다. 에러값인 'err'가 0인 것으로 보아 정상적으로 작동하는 것을 확인할 수 있다. 파이프라인을 적용하지 않아 rstn이 1이 되고 난 뒤 한 클럭 사이클이 지나고부터 바로 결과가 출력되는 것을 볼 수 있다.

c. Schematics



위 그림들은 합성 후의 schematic으로 critical path를 빨간색으로 나타냈다. 21개의 가산기는 이전까지 설계했던 RCA와는 다르게, 22비트 cin과 22비트 cout 포트를 가지고 있다. 각 cin은 대응되는 가산기 모듈 내의 FA에 입력되고, cout은 대응되는 FA의 cout 핀과 연결되어 있다. 따라서 carry propagation이 없고, critical path가 하나의 FA인 것을 볼 수 있다. 또한 이 가산기에 입력될 a, b를 구하기 위해 22개의 AND 게이트를 모아둔 and_gates 모듈도 확인할 수 있다. CSM_RCA 모듈의 critical path도 확인할 수 있다. 시작은 최초 and_gates 모듈에서 시작하여, 21개의 22-bit adder를 거치고, 마지막으로 vector merge adder (22-bit RCA)를 거치는 것을 볼 수 있다. Vector merge adder는 carry propagation이 있으므로 22FA의 딜레이를 가진다. 그래서 전체 CPD를 보면 $1AND + 21FA + 22FA = 1AND + 43FA$ 의 딜레이를 가진다고 볼 수 있다.

d. PPA Results

i. Timing

아래 첨부한 그림은 0.1ns씩 클럭 주기를 조정하여 최종적으로 17.10ns 클럭에서 합성했을 때의 timing report이다. Slack이 음수가 되면서 타이밍을 위반한다.

csm_rca/vec_merge/sum[21] (rca_22bit)	0.00	16.95 t
csm_rca/mul[43] (CSM_RCA_22bit)	0.00	16.95 f
U115/Q (and2s2)	0.23	17.17 f
mul_reg[43]/DIN (dffs1)	0.00	17.17 f
data arrival time		17.17
clock clk (rise edge)	17.10	17.10
clock network delay (ideal)	0.00	17.10
mul_reg[43]/CLK (dffs1)	0.00	17.10 r
library setup time	-0.32	16.78
data required time		16.78

data required time		16.78
data arrival time		-17.17

slack (VIOLATED)		-0.39

아래의 그림은 다시 클럭 주기를 17.20ns로 설정하고 합성했을 때의 timing report이다. Slack이 0이 되면서 타이밍을 만족하는 것을 관찰할 수 있다.

왼쪽의 그림은 클럭 주기가 17.10ns일 때의 면적이다. 오른쪽의 그림은 클럭 주기가 17.20ns로 타이밍 조건을 충족했을 때의 면적이다. 클럭 주기가 늘어났을 때 면적이 $688\mu\text{m}^2$ 감소한 것을 볼 수 있다. 지연을 줄이기 위해 더 큰 면적의 소자들을 사용한 것으로 볼 수 있다.

iii. Power

Global Operating Voltage = 2.25
Power-specific unit information :
Voltage Units = 1V
Capacitance Units = 1,000,000pf
Time Units = 1ns
Dynamic Power Units = 1mW (derived from V,C,T units)
Leakage Power Units = 1pW

Cell Internal Power = 6.0225 mW (69%)
Net Switching Power = 2.6766 mW (31%)

Total Dynamic Power = 8.6992 mW (100%)
Cell Leakage Power = 7.1664 uW

Global Operating Voltage = 2.25
Power-specific unit information :
Voltage Units = 1V
Capacitance Units = 1,000,000pf
Time Units = 1ns
Dynamic Power Units = 1mW (derived from V,C,T units)
Leakage Power Units = 1pW

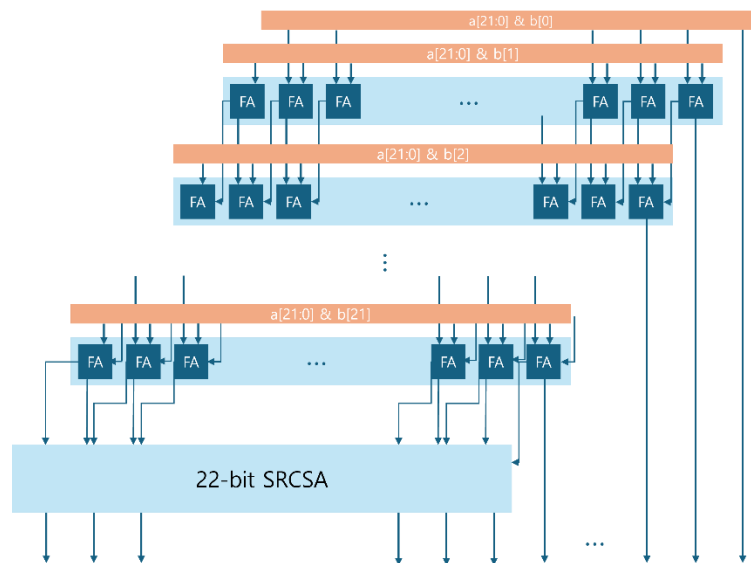
Cell Internal Power = 5.7057 mW (69%)
Net Switching Power = 2.5964 mW (31%)

Total Dynamic Power = 8.3021 mW (100%)
Cell Leakage Power = 6.7020 uW

왼쪽은 클럭 주기가 17.10ns일 때의 파워, 오른쪽은 17.20ns로 타이밍을 충족했을 때의 파워이다. 오른쪽의 total dynamic power가 더 작는데, 이는 클럭주기가 높을수록(주파수가 낮을수록) 단위시간당 더 적은 계산을 수행하기 때문에 발생하는 차이라고 볼 수 있다.

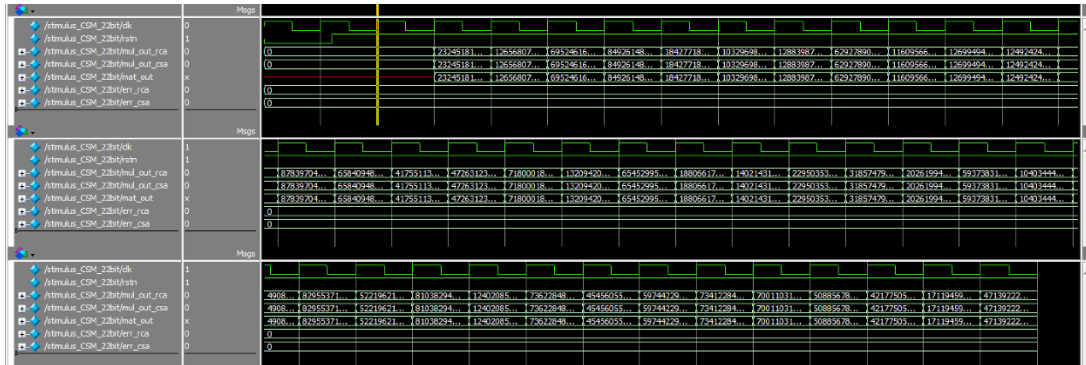
2. 22-bit Carry Save Multiplier with SRCSA

a. Architecture



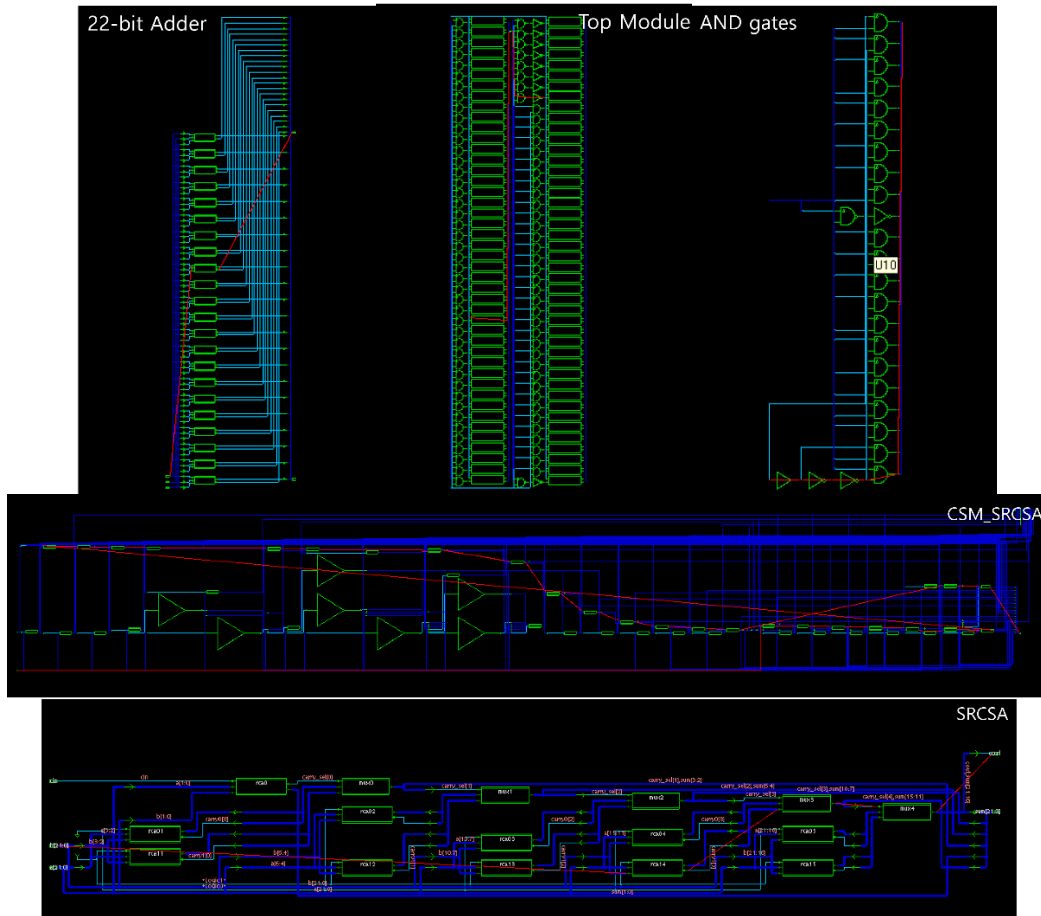
전체적인 구조 자체는 RCA를 썼을 때와 유사하다. 차이가 있다면 vector merging adder의 자리에 22-bit RCA 대신 22-bit SRCSA가 존재한다는 것이다. RCA를 썼을 때의 딜레이가 1AND + 43FA였다면, 이번에는 SRCSA를 쓰기 때문에 3주차 실험에서 얻었던 결과를 적용하면, 총 1AND + 21FA + (3FA+4MUX) = 1AND + 24FA + 4MUX의 딜레이를 갖게 된다. FA와 MUX의 딜레이가 같다고 가정했기 때문에 1AND+28FA의 딜레이를 갖는다고 볼 수도 있다. 따라서 15FA만큼의 딜레이 감소를 기대할 수 있고, AND의 딜레이가 FA의 딜레이보다 훨씬 작을 것이므로 딜레이가 대략 2/3정도로 줄어든 것이라고 생각해볼 수 있다.

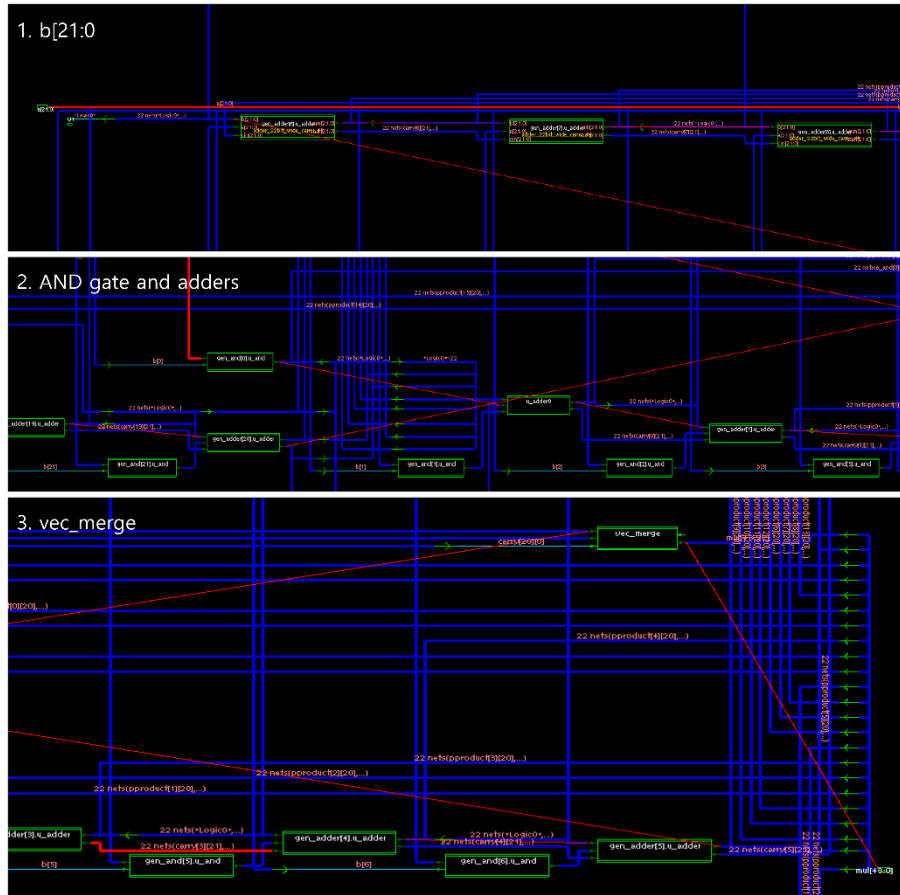
b. Pre-Synthesis Simulation



RCA를 이용했을 때와 같은 테스트벤치를 사용했다. SRCSA를 이용한 곱셈기의 결과인 `mul_out_csa`가 정답과 같은 파형을 보이고 있으며, `err_csa`도 0이므로 제대로 기능한다고 볼 수 있다. 이번에도 파이프라인이 적용되지 않았으므로 `rstn=1`이 되고 첫번째 posedge clk이 발생한 뒤로부터 한 클럭사이클이 지났을 때부터 결과가 출력되고 있다.

c. Schematics





위 그림들은 합성 후의 schematic으로 critical path를 빨간색으로 나타냈다. RCA를 이용했을 때와 비슷한 결과가 나왔다. 21개의 가산기는 carry propagation이 없으므로, critical path가 하나의 FA인 것을 볼 수 있고 AND 게이트를 모아둔 and_gates 모듈도 확인할 수 있다. CSM_SRCSCA 모듈의 critical path도 확인할 수 있다. Flyline이 복잡하긴 하지만, 시작은 최초 b포트와 and_gates 모듈에서 시작하여, 21개의 22-bit adder를 거치고, 마지막으로 vector merge adder (22-bit SRCSCA)를 거치게 되는데 이 경로는 RCA를 사용했을 때와 완전히 동일하다. Vector merge adder는 5FA+2MUX (7FA)의 딜레이를 가지므로 전체 CPD를 보면 $1AND + 21FA + 7FA = 1AND + 28FA$ 의 딜레이를 가진다.

d. PPA Results

i. Timing

아래 첨부한 그림은 0.1ns씩 클럭 주기를 조정하여 최종적으로 14.30ns 클럭에서 합성했을 때의 timing report이다. Slack이 음수가 되면서 타이밍을 위반한다.

csu_rca/vec_merge/mux3/sel (mux_param_WIDTH6)	0.00	13.69	r
csu_rca/vec_merge/mux3/U8/Q (mxi21s3)	0.38	14.08	f
csu_rca/vec_merge/mux3/out[5] (mux_param_WIDTH6)	0.00	14.08	f
csu_rca/vec_merge/mux4/sel (mux_param_WIDTH7)	0.00	14.08	f
csu_rca/vec_merge/mux4/U8/Q (mxi21s3)	0.30	14.37	f
csu_rca/vec_merge/mux4/out[1] (mux_param_WIDTH7)	0.00	14.37	f
csu_rca/vec_merge/sum[17] (sqrt_csa)	0.00	14.37	f
csu_rca/mul[39] (CSM_SRCsa_22bit)	0.00	14.37	f
U186/Q (rnd2s3)	0.10	14.47	r
U93/Q (i1s3)	0.08	14.55	f
mul_reg[39]/DIN (dffs1)	0.00	14.55	f
data arrival time		14.55	
clock clk (rise edge)	14.30	14.30	
clock network delay (ideal)	0.00	14.30	
mul_reg[39]/CLK (dffs1)	0.00	14.30	r
library setup time	-0.31	13.99	
data required time		13.99	
data required time		13.99	
data arrival time		-14.55	
slack (VIOLATED)		-0.56	

아래의 그림은 다시 클럭 주기를 14.40ns로 설정하고 합성했을 때의 timing report이다. Slack이 0이 되면서 타이밍을 만족하는 것을 관찰할 수 있다.

Point	Incr	Path	csu_rca/gen_adder[9].u_adder/fa_gen[14].u_fa/U2/Q (xor2s3)	csu_rca/gen_adder[17].u_adder/fa_gen[8].u_fa/b (fa_122)		
clock clk (rise edge)	0.00	0.00	0.29	5.53	r	9.44
clock network delay (ideal)	0.00	0.00	0.13	5.66	f	9.69
b_reg_reg[1]/CLK (dffs2)	0.00	0.00	0.14	5.81	r	10.03
b_reg_reg[1]/Q (dffs2)	0.40	0.40	0.00	5.81	r	10.03
csu_rca/b[1] (CSM_SRCsa_22bit)	0.00	0.40	0.00	5.81	r	10.03
csu_rca/gen_and[1].u_and/b (and_gates_21)	0.00	0.40	0.00	5.81	r	10.03
csu_rca/gen_and[1].u_and/U19/Q (rnd2s1)	0.23	0.63	0.00	5.81	r	10.03
csu_rca/gen_and[1].u_and/U19/Q (rnd2s1)	0.20	0.83	0.00	5.81	r	10.03
csu_rca/gen_and[1].u_and/out[20] (and_gates_21)	0.00	0.83	0.00	5.81	r	10.03
csu_rca/u_adder0/a[20] (adder_22bit_wide_carry_0)	0.00	0.83	0.00	5.81	r	10.03
csu_rca/u_adder0/fa_gen[20].u_fa/a (fa_484)	0.00	1.05	0.00	5.81	r	10.03
csu_rca/u_adder0/fa_gen[20].u_fa/U1/Q (xor2s3)	0.22	1.05	0.00	5.81	r	10.03
csu_rca/u_adder0/fa_gen[20].u_fa/U1/Q (xor2s3)	0.14	1.19	0.00	5.81	r	10.03
csu_rca/u_adder0/fa_gen[20].u_fa/cout (fa_484)	0.00	1.19	0.00	5.81	r	10.03
csu_rca/u_adder0/cout[20] (adder_22bit_wide_carry_0)	0.00	1.19	0.00	5.81	r	10.03
csu_rca/gen_adder[1].u_adder/cin[20] (adder_22bit_wide_carry_20)	0.00	1.19	0.00	5.81	r	10.03
csu_rca/gen_adder[1].u_adder/fa_gen[20].u_fa/cin (fa_462)	0.00	1.19	0.00	5.81	r	10.03
csu_rca/gen_adder[1].u_adder/fa_gen[20].u_fa/U7/Q (xor2s3)	0.00	1.19	0.00	5.81	r	10.03
csu_rca/gen_adder[1].u_adder/fa_gen[20].u_fa/cin (fa_462)	0.47	1.66	0.00	5.81	r	10.03
csu_rca/gen_adder[1].u_adder/sun[20] (adder_22bit_wide_carry_20)	0.00	1.66	0.00	5.81	r	10.03
csu_rca/gen_adder[1].u_adder/b[19] (adder_22bit_wide_carry_19)	0.00	1.66	0.00	5.81	r	10.03
csu_rca/gen_adder[2].u_adder/b[19] (adder_22bit_wide_carry_19)	0.00	1.66	0.00	5.81	r	10.03
csu_rca/gen_adder[2].u_adder/fa_gen[19].u_fa/b (fa_441)	0.00	1.66	0.00	5.81	r	10.03
csu_rca/gen_adder[2].u_adder/sun[20] (adder_22bit_wide_carry_20)	0.00	1.66	0.00	5.81	r	10.03
csu_rca/gen_adder[2].u_adder/b[19] (adder_22bit_wide_carry_19)	0.00	1.66	0.00	5.81	r	10.03
csu_rca/gen_adder[2].u_adder/fa_gen[19].u_fa/b (fa_441)	0.00	1.66	0.00	5.81	r	10.03
csu_rca/gen_adder[2].u_adder/fa_gen[19].u_fa/U3/Q (xor2s3)	0.00	1.66	0.00	5.81	r	10.03
csu_rca/gen_adder[2].u_adder/fa_gen[19].u_fa/U4/Q (rnd2s2)	0.25	1.92	0.00	5.81	r	10.03
csu_rca/gen_adder[2].u_adder/fa_gen[19].u_fa/U4/Q (rnd2s2)	0.24	2.16	0.00	5.81	r	10.03
csu_rca/gen_adder[2].u_adder/cout[12] (adder_22bit_wide_carry_9)	0.00	2.16	0.00	5.81	r	10.03
csu_rca/gen_adder[2].u_adder/cout[19] (adder_22bit_wide_carry_19)	0.00	2.16	0.00	5.81	r	10.03
csu_rca/gen_adder[3].u_adder/cin[19] (adder_22bit_wide_carry_18)	0.00	2.16	0.00	5.81	r	10.03
csu_rca/gen_adder[3].u_adder/cin[19] (adder_22bit_wide_carry_18)	0.00	2.16	0.00	5.81	r	10.03
csu_rca/gen_adder[3].u_adder/fa_gen[19].u_fa/cin (fa_419)	0.00	2.16	0.00	5.81	r	10.03
csu_rca/gen_adder[3].u_adder/fa_gen[19].u_fa/U3/Q (xor2s3)	0.00	2.16	0.00	5.81	r	10.03
csu_rca/gen_adder[3].u_adder/fa_gen[19].u_fa/cin (fa_419)	0.37	2.53	0.00	5.81	r	10.03
csu_rca/gen_adder[3].u_adder/sun[19] (adder_22bit_wide_carry_18)	0.00	2.53	0.00	5.81	r	10.03
csu_rca/gen_adder[3].u_adder/b[18] (adder_22bit_wide_carry_17)	0.00	2.53	0.00	5.81	r	10.03
csu_rca/gen_adder[4].u_adder/fa_gen[18].u_fa/b (fa_398)	0.00	2.53	0.00	5.81	r	10.03
csu_rca/gen_adder[4].u_adder/fa_gen[18].u_fa/U1/Q (xor2s3)	0.00	2.53	0.00	5.81	r	10.03
csu_rca/gen_adder[4].u_adder/fa_gen[18].u_fa/U4/Q (xor2s3)	0.25	2.78	0.00	5.81	r	10.03
csu_rca/gen_adder[6].u_adder/sun[17] (adder_22bit_wide_carry_15)	0.34	3.12	0.00	5.81	r	10.03
csu_rca/gen_adder[7].u_adder/b[16] (adder_22bit_wide_carry_14)	0.00	4.06	0.00	5.81	r	10.03
csu_rca/gen_adder[7].u_adder/fa_gen[16].u_fa/b (fa_334)	0.00	4.06	0.00	5.81	r	10.03
csu_rca/gen_adder[7].u_adder/fa_gen[16].u_fa/U1/Q (xor2s3)	0.25	4.31	0.00	5.81	r	10.03
csu_rca/gen_adder[7].u_adder/fa_gen[16].u_fa/U3/Q (xor2s3)	0.34	4.65	0.00	5.81	r	10.03
csu_rca/gen_adder[7].u_adder/sun[16] (adder_22bit_wide_carry_14)	0.00	4.65	0.00	5.81	r	10.03
csu_rca/gen_adder[8].u_adder/b[15] (adder_22bit_wide_carry_13)	0.00	4.65	0.00	5.81	r	10.03
csu_rca/gen_adder[8].u_adder/fa_gen[15].u_fa/b (fa_313)	0.00	4.65	0.00	5.81	r	10.03
csu_rca/gen_adder[8].u_adder/fa_gen[15].u_fa/U1/Q (xor2s3)	0.25	4.90	0.00	5.81	r	10.03
csu_rca/gen_adder[8].u_adder/fa_gen[15].u_fa/U2/Q (xor2s3)	0.34	5.24	0.00	5.81	r	10.03
csu_rca/gen_adder[8].u_adder/sun[15] (adder_22bit_wide_carry_13)	0.00	5.24	0.00	5.81	r	10.03
csu_rca/gen_adder[9].u_adder/b[14] (adder_22bit_wide_carry_12)	0.00	5.24	0.00	5.81	r	10.03
csu_rca/gen_adder[9].u_adder/fa_gen[14].u_fa/b (fa_292)	0.00	5.24	0.00	5.81	r	10.03

Critical path는 RCA에서와 비슷하게 and 게이트로부터 출발해서, 21개의 모든 adder를 거쳐 최종적으로 SRCsa로 만들어진 vector merging adder까지만 것을 볼 수 있다. 타이

밍 리포트에 모듈 이름이 csm_rca라고 되어있는데, 이는 합성 당시 베릴로그 코드에서 csm_srcsa가 아닌 csm_rca라는 이름으로 instantiate되어 있어서 그렇다. 실제 사용된 모듈은 SRCSA가 맞고, 그로 인해서 딜레이가 감소한 것도 볼 수 있다. 최소 클럭 주기는 14.40ns이고, F_{max} 는 $\frac{1}{14.4 \times 10^{-9}} = 69.4\text{MHz}$ 이다. 속도가 19.4% 빨라졌는데, Architecture 항목에서 했던 예측과 완전히 일치하지는 않지만, SRCSA를 이용함으로써 속도 측면에서 상당한 이득을 봤음을 알 수 있다.

ii. Area

```
*****
Report : area
Design : CSM_SRCsa_22bit_top
Version: Z-2007.03-SP4
Date   : Fri Apr 10 12:18:05 2006
*****

Library(s) Used:

lec25dscc25_SS (File: /home/admin/lib/lec25/lec25dscc25_SS.db)

Number of ports:      90
Number of nets:      277
Number of cells:      186
Number of references: 8

Combinational area: 209101.938747
Noncombinational area: 14598.144043
Net Interconnect area: undefined (No wire load specified)

Total cell area: 223700.078125
Total area: undefined

Library(s) Used:

lec25dscc25_SS (File: /home/admin/lib/lec25/lec25dscc25_SS.db)

Number of ports:      90
Number of nets:      275
Number of cells:      186
Number of references: 8

Combinational area: 207973.909851
Noncombinational area: 14598.144043
Net Interconnect area: undefined (No wire load specified)

Total cell area: 222572.046875
Total area: undefined
```

왼쪽의 그림은 클럭 주기가 14.30ns일 때의 면적이다. 오른쪽의 그림은 클럭 주기가 14.40ns로 타이밍 조건을 충족했을 때의 면적이다. 클럭 주기가 늘어났을 때 면적은 $1128\mu\text{m}^2$ 감소했다.

iii. Power

```
Operating Conditions: nom_pvt Library: lec25dscc25_SS
Wire Load Model Mode: top

Global Operating Voltage = 2.25
Power-specific unit information :
  Voltage Units = 1V
  Capacitance Units = 1.000000pf
  Time Units = 1ns
  Dynamic Power Units = 1mW (derived from V,C,T units)
  Leakage Power Units = 1pW

Cell Internal Power = 7.8549 mW (70%)
Net Switching Power = 3.4267 mW (30%)
-----
Total Dynamic Power = 11.2816 mW (100%)
Cell Leakage Power = 381.3580 uW

Global Operating Voltage = 2.25
Power-specific unit information :
  Voltage Units = 1V
  Capacitance Units = 1.000000pf
  Time Units = 1ns
  Dynamic Power Units = 1mW (derived from V,C,T units)
  Leakage Power Units = 1pW

Cell Internal Power = 7.6582 mW (69%)
Net Switching Power = 3.3992 mW (31%)
-----
Total Dynamic Power = 11.0574 mW (100%)
Cell Leakage Power = 7.4905 uW
```

왼쪽은 클럭 주기가 14.30ns일 때의 파워, 오른쪽은 14.40ns로 타이밍을 충족했을 때의 파워이다. 클럭 주기가 14.40ns일 때 total dynamic power가 더 낮는데, 이것은 클럭주기가 높을수록 단위시간당 더 적은 계산을 수행하기 때문에 생기는 차이라고 볼 수 있다.

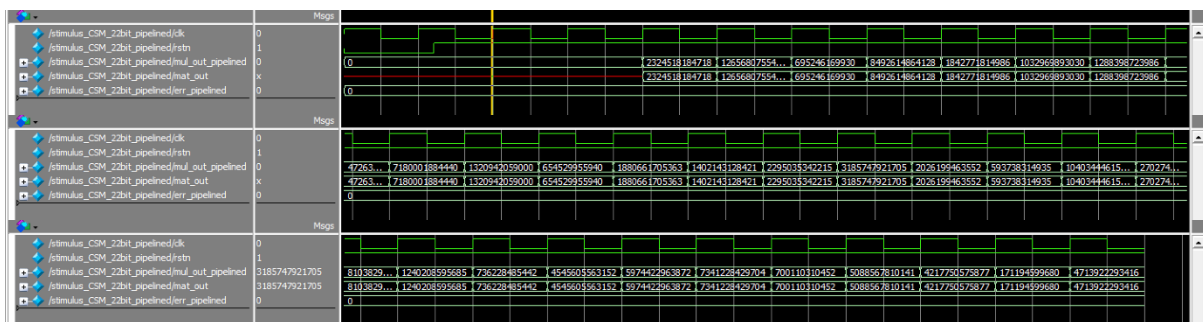
3. Pipelining a 22-bit Carry Save Multiplier with RCA

a. Architectural Considerations

처음에는 21FA와 22-bit RCA 사이에 파이프라인을 추가하고자 했다. 이렇게 되면 1AND+21FA와 22FA로 critical path가 거의 이등분되기 때문에 가장 좋은 결과를 얻을 수 있을 것이라고 생각했다. 하지만 이런 가정이 22-bit RCA를 이용한 CSM의 타이밍 리포트 결과와 일치하지 않는 면이 있어서 타이밍 리포트를 바탕으로 파이프라인 위치를 수정했다.

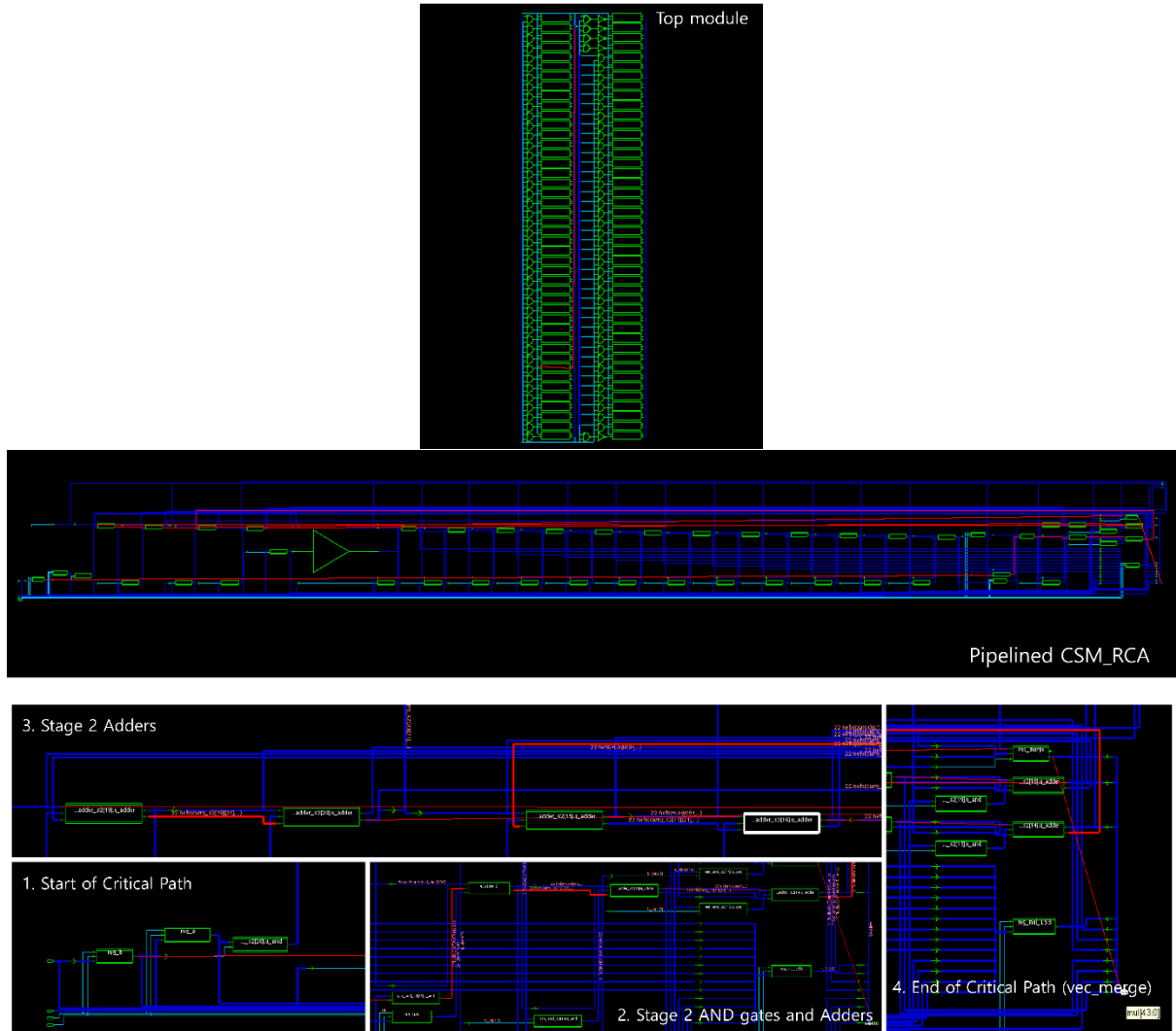
1번 실험에서 얻었던 minimum clock cycle은 17.2ns였고, clock-to-q delay와 setup time이 0.31ns로 일치했다. 따라서 clock cycle의 정중앙인 8.6ns 부근에 파이프라인을 추가해야 한다. 타이밍 리포트를 자세히 보면 12번째 22비트 가산기인 gen_adder[11]이 대략 8.6ns 부근에 위치하는 것을 알 수 있다. 그래서 gen_adder[11]과 gen_adder[12] 사이에 파이프라인을 추가하는 선택을 했다.

b. Pre-Synthesis Simulation



위는 시뮬레이션 파형의 처음, 중간, 끝부분을 나타낸 것이다. Mul_out_pipelined의 파형과 mat_out이 완전히 일치하고 있다. 에러값인 'err_pipelined'가 0인 것으로 보아 정상적으로 작동하는 것을 확인할 수 있다. 이 파형에서는 reset이 1이 된 이후, 총 두 번의 clock rising edge가 지나고 세번째 posedge에서 결과가 출력되고 있다. 이것은 2-stage pipeline을 적용한 설계 때문이다. 모든 파이프라인 단계를 통과하기까지 두 클럭 사이클이 걸리기 때문에 초반 latency가 증가하는 것을 확인할 수 있다.

c. Schematics



위 그림들은 합성 후의 Schematic이다. CSM_RCA 모듈 내부의 critical path가 복잡해서 확대하여 자세하게 표시했다. 먼저 reg_b로부터 시작한다. reg_b는 입력으로 받았던 b값의 9 MSB를 저장하는 9비트 레지스터이다. 이를 저장하여 stage 2에서 이용할 수 있도록 하는 역할을 수행한다. 그 이후 stage 2의 and게이트를 지나 가산기에 입력이 될 수 있는 값이 되고, u_adder12부터 시작하여 두번째 파이프라인 단계의 가산기들을 거쳐간다. 마지막으로 RCA인 vector merging adder를 지나는 경로를 밝게 된다.

d. PPA Results

i. Timing

아래 첨부한 그림은 0.1ns씩 클럭 주기를 조정하여 최종적으로 10.70ns 클럭에서 합성했을 때의 timing report이다. Slack이 음수가 되면서 타이밍을 위반한다.

csu_rca/vec_merge/fa_gen[20].u_fa/cin (fa_2)	0.00	9.76 r
csu_rca/vec_merge/fa_gen[20].u_fa/U2/Q (nnd2s3)	0.10	9.86 f
csu_rca/vec_merge/fa_gen[20].u_fa/U1/Q (nnd2s3)	0.12	9.98 r
csu_rca/vec_merge/fa_gen[20].u_fa/cout (fa_2)	0.00	9.98 r
csu_rca/vec_merge/fa_gen[21].u_fa/cin (fa_1)	0.00	9.98 r
csu_rca/vec_merge/fa_gen[21].u_fa/U1/Q (xor2s3)	0.28	10.26 r
csu_rca/vec_merge/fa_gen[21].u_fa/sum (fa_1)	0.00	10.26 r
csu_rca/vec_merge/sum[21] (rca_22bit)	0.00	10.26 r
csu_rca/mul[43] (CSU_RCA_pipelined_new)	0.00	10.26 r
U92/Q (nnd2s3)	0.09	10.35 f
U91/Q (i1s3)	0.08	10.43 r
mul_reg[43]/DIN (dffs1)	0.00	10.43 r
data arrival time		10.43
clock clk (rise edge)	10.70	10.70
clock network delay (ideal)	0.00	10.70
mul_reg[43]/CLK (dffs1)	0.00	10.70 r
library setup time	-0.30	10.40
data required time		10.40
data required time		10.40
data arrival time		-10.43
slack (VIOLATED)		-0.03

아래의 그림은 다시 클럭 주기를 10.80ns로 설정하고 합성했을 때의 timing report이다. Slack이 0이 되면서 타이밍을 만족하는 것을 관찰할 수 있다.

clock clk (rise edge)	0.00	0.00	csu_rca/gen_adder_s2[16].u_adder/fa_gen[15].u_fa/U1/Q (xor2s1)	0.00	7.27 r
clock network delay (ideal)	0.00	0.00	csu_rca/gen_adder_s2[16].u_adder/fa_gen[15].u_fa/U1/Q (xor2s1)	0.40	3.94 r
csu_rca/reg_b/a_reg[0]/CLK (dffs2)	0.00	0.00	csu_rca/gen_adder_s2[16].u_adder/fa_gen[15].u_fa/U2/Q (xor2s1)	0.44	4.38 r
csu_rca/reg_b/q_reg[0]/Q (dffs2)	0.32	0.32	csu_rca/gen_adder_s2[16].u_adder/sum[15] (adder_22bit_wide_carry_5)	0.00	4.38 r
csu_rca/reg_b/U2/Q (i1s6)	0.17	0.49	csu_rca/gen_adder_s2[17].u_adder/b[14] (adder_22bit_wide_carry_4)	0.00	4.38 r
csu_rca/reg_b/a[0] (IFF_para_KIDTH9)	0.00	0.49	csu_rca/gen_adder_s2[17].u_adder/fa_gen[14].u_fa/b (fa_96)	0.00	4.67 r
csu_rca/gen_and_s2[13].u_and/b (and_gates_9)	0.00	0.49	csu_rca/gen_adder_s2[17].u_adder/fa_gen[14].u_fa/U5/Q (xor2s2)	0.29	5.04 r
csu_rca/gen_and_s2[13].u_and/U2/Q (and2s1)	0.21	0.80	csu_rca/gen_adder_s2[17].u_adder/fa_gen[14].u_fa/U1/Q (xor2s2)	0.37	5.04 r
csu_rca/gen_and_s2[13].u_and/out[18] (and_gates_9)	0.00	0.80	csu_rca/gen_adder_s2[17].u_adder/fa_gen[14].u_fa/sum (fa_96)	0.00	5.04 r
csu_rca/u_adder12/al[18] (adder_22bit_wide_carry_9)	0.00	0.80	csu_rca/gen_adder_s2[18].u_adder/b[13] (adder_22bit_wide_carry_3)	0.00	5.04 r
csu_rca/u_adder12/fa_gen[18].u_fa/a (fa_202)	0.00	0.80	csu_rca/gen_adder_s2[18].u_adder/fa_gen[13].u_fa/b (fa_75)	0.00	5.04 r
csu_rca/u_adder12/fa_gen[18].u_fa/U2/Q (xor2s2)	1.17	1.48	csu_rca/gen_adder_s2[18].u_adder/fa_gen[13].u_fa/U2/Q (xor2s2)	0.28	5.32 r
csu_rca/u_adder12/fa_gen[18].u_fa/U1/Q (xor2s2)	0.31	1.48	csu_rca/gen_adder_s2[18].u_adder/fa_gen[13].u_fa/U5/Q (xor2s2)	0.44	5.76 r
csu_rca/u_adder12/sum[18] (adder_22bit_wide_carry_9)	0.00	1.48	csu_rca/gen_adder_s2[18].u_adder/fa_gen[13].u_fa/U1/Q (xor2s1)	0.28	5.76 r
csu_rca/gen_adder_s2[13].u_adder/b[17] (adder_22bit_wide_carry_8)	0.00	1.48	csu_rca/gen_adder_s2[19].u_adder/b[12] (adder_22bit_wide_carry_2)	0.00	5.76 r
csu_rca/gen_adder_s2[13].u_adder/fa_gen[17].u_fa/b (fa_181)	0.00	1.48	csu_rca/gen_adder_s2[19].u_adder/fa_gen[12].u_fa/b (fa_54)	0.00	5.76 r
csu_rca/gen_adder_s2[13].u_adder/fa_gen[17].u_fa/U1/Q (xor2s1)	0.40	1.88	csu_rca/gen_adder_s2[19].u_adder/fa_gen[12].u_fa/U1/Q (xor2s1)	0.32	6.09 f
csu_rca/gen_adder_s2[13].u_adder/fa_gen[17].u_fa/U2/Q (xor2s1)	0.23	2.33	csu_rca/gen_adder_s2[19].u_adder/fa_gen[12].u_fa/U2/Q (xor2s2)	0.31	6.40 f
csu_rca/gen_adder_s2[13].u_adder/sum[17] (adder_22bit_wide_carry_8)	0.00	2.33	csu_rca/gen_adder_s2[19].u_adder/sum[12] (adder_22bit_wide_carry_2)	0.00	6.40 f
csu_rca/gen_adder_s2[14].u_adder/b[16] (adder_22bit_wide_carry_7)	0.00	2.33	csu_rca/gen_adder_s2[20].u_adder/b[11] (adder_22bit_wide_carry_1)	0.00	6.40 f
csu_rca/gen_adder_s2[14].u_adder/fa_gen[16].u_fa/b (fa_160)	0.23	2.33	csu_rca/gen_adder_s2[20].u_adder/fa_gen[11].u_fa/b (fa_33)	0.00	6.40 f
csu_rca/gen_adder_s2[14].u_adder/fa_gen[16].u_fa/U2/Q (xor2s1)	0.23	2.33	csu_rca/gen_adder_s2[20].u_adder/fa_gen[11].u_fa/U1/Q (xor2s1)	0.40	6.80 r
csu_rca/gen_adder_s2[14].u_adder/fa_gen[16].u_fa/U5/Q (and2s1)	0.23	2.61	csu_rca/gen_adder_s2[20].u_adder/fa_gen[11].u_fa/U4/Q (xor2s1)	0.48	7.27 r
csu_rca/gen_adder_s2[14].u_adder/fa_gen[16].u_fa/U3/Q (fa_160)	0.31	2.93	csu_rca/gen_adder_s2[20].u_adder/sum[11] (adder_22bit_wide_carry_1)	0.00	7.27 r
csu_rca/gen_adder_s2[14].u_adder/fa_gen[16].u_fa/cout (fa_160)	0.31	3.23	csu_rca/gen_adder_s2[21].u_adder/b[10] (adder_22bit_wide_carry_0)	0.00	7.27 r
csu_rca/gen_adder_s2[14].u_adder/cout[16] (adder_22bit_wide_carry_7)	0.00	3.23	csu_rca/gen_adder_s2[21].u_adder/fa_gen[10].u_fa/b (fa_33)	0.00	7.27 r
csu_rca/gen_adder_s2[15].u_adder/cin[16] (adder_22bit_wide_carry_6)	0.00	3.23	csu_rca/gen_adder_s2[21].u_adder/fa_gen[10].u_fa/U5/Q (xor2s2)	0.29	5.04 r
csu_rca/gen_adder_s2[15].u_adder/fa_gen[16].u_fa/cin (fa_138)	0.00	3.23	csu_rca/gen_adder_s2[21].u_adder/fa_gen[10].u_fa/U1/Q (xor2s2)	0.37	5.04 r
csu_rca/gen_adder_s2[15].u_adder/fa_gen[16].u_fa/U2/Q (xor2s1)	0.31	3.54	csu_rca/gen_adder_s2[21].u_adder/fa_gen[10].u_fa/sum (fa_33)	0.00	5.04 r
csu_rca/gen_adder_s2[15].u_adder/fa_gen[16].u_fa/sum (fa_138)	0.00	3.54	csu_rca/gen_adder_s2[21].u_adder/sum[10] (adder_22bit_wide_carry_0)	0.00	5.04 r
csu_rca/gen_adder_s2[15].u_adder/sum[16] (adder_22bit_wide_carry_5)	0.00	3.54	csu_rca/gen_adder_s2[21].u_adder/sum[10] (adder_22bit_wide_carry_0)	0.00	5.04 r
csu_rca/gen_adder_s2[16].u_adder/b[15] (adder_22bit_wide_carry_5)	0.00	3.54	csu_rca/gen_adder_s2[21].u_adder/sum[10] (adder_22bit_wide_carry_0)	0.00	5.04 r
csu_rca/gen_adder_s2[16].u_adder/fa_gen[15].u_fa/b (fa_117)	0.00	3.54	csu_rca/gen_adder_s2[21].u_adder/sum[10] (adder_22bit_wide_carry_0)	0.00	5.04 r

Schematic 항목과는 동일하게 critical path는 reg_b로부터 시작해, stage 2의 and 게이트인 gen_and_s2를 지나서 stage 2의 가산기 u_adder12와 gen_adder_s2를 지나 최종적으로 vec_merge 유닛까지 지나는 것을 확인할 수 있다. 최소 클럭 주기는 10.80ns이고, F_{max} 는 $\frac{1}{10.80 \times 10^{-9}} = 92.6\text{MHz}$ 이다. 파이프라인을 적용하지 않았을 때는 58.1MHz의 주파수로 동작을 했다. 따라서 파이프라인 적용을 통해 59.4% 더 빠른 속도로 회로가 작동할 수 있게 됐다.

ii. Area

Library(s) Used:		Library(s) Used:	
lec25dsc25_SS (File: /home/admin/lib/lec25/lec25dsc25_SS.db)		lec25dsc25_SS (File: /home/admin/lib/lec25/lec25dsc25_SS.db)	
Number of ports:	90	Number of ports:	90
Number of nets:	266	Number of nets:	266
Number of cells:	177	Number of cells:	177
Number of references:	4	Number of references:	4
Combinational area:	198203,107403	Combinational area:	197431,726971
Noncombinational area:	25596,521133	Noncombinational area:	25546,754120
Net Interconnect area:	undefined (No wire load specified)	Net Interconnect area:	undefined (No wire load specified)
Total cell area:	223799,625000	Total cell area:	222978,484375
Total area:	undefined	Total area:	undefined

왼쪽의 그림은 클럭 주기가 10.70ns일 때의 면적이다. 오른쪽의 그림은 클럭 주기가 10.80ns로 타이밍 조건을 충족했을 때의 면적이다. 클럭 주기를 늘렸을 때 면적이 줄어드는 것을 볼 수 있다.

iii. Power

Global Operating Voltage = 2.25		Global Operating Voltage = 2.25	
Power-specific unit information :		Power-specific unit information :	
Voltage Units = 1V		Voltage Units = 1V	
Capacitance Units = 1.000000pf		Capacitance Units = 1.000000pf	
Time Units = 1ns		Time Units = 1ns	
Dynamic Power Units = 1mW (derived from V,C,T units)		Dynamic Power Units = 1mW (derived from V,C,T units)	
Leakage Power Units = 1pW		Leakage Power Units = 1pW	
Cell Internal Power	= 10,8821 mW (72%)	Cell Internal Power	= 10,6580 mW (72%)
Net Switching Power	= 4,2901 mW (28%)	Net Switching Power	= 4,2197 mW (28%)
Total Dynamic Power	= 15,1722 mW (100%)	Total Dynamic Power	= 14,8777 mW (100%)
Cell Leakage Power	= 7,5184 uW	Cell Leakage Power	= 7,3931 uW

왼쪽은 클럭 주기가 10.70ns일 때의 파워, 오른쪽은 10.80ns로 타이밍을 충족했을 때의 파워이다. 오른쪽의 경우가 파워가 낮는데, 클럭주기가 낮을수록 단위시간당 더 많은 계산을 수행하기 때문에 생기는 차이라고 볼 수 있다.

4. Discussion

a. Time and Area complexity of a CSM

CSM 구조 같은 최적화 없이 Naïve multiplier을 구현하게 되면 시간 복잡도와 공간복잡도는 $O(n^2)$ 이 된다. n비트 곱셈을 구현하기 위해 n^2 개의 가산기가 필요하고, 하나의 가산기는 이전 단계의 가산기의 carry out을 기다려야 하기 때문이다. 하지만 CSM구조를 택하게 되면 CPD는 n^2 이 아닌 $(n-1)+n=2n-1$ 에 비례하게 된다. 물론 실질적으로는 vector merge adder 이전 n-1개의 n비트 가산기 사이의 배선 등의 지연으로 인해 $2n-1$ 보다 지연이 더 커지게 되겠지만, 선형적인 비례를 보인다는 것을 이 실험을 통해 알 수 있었다.

22-bit RCA를 구현했을 때 최소 클럭 주기가 5.66ns였는데, 만약 naïve multiplier를 설계했다면 가산기 구조만으로 CPD가 이것의 제곱으로 늘어날 수 있고, 추가적인 AND게이트와 배선으로 인해 딜레이는 더 커질 수 있었을 것이라고 생각해볼 수 있다. Carry의 전

달 경로를 살짝 바꾸는 것만으로 17.20ns의 CPD를 달성했다는 것은 인상 깊은 결과이다.

b. Comparison of Timing, Area and Power

	Naïve RCA	SRCSA	Pipelined RCA
F_max	58.1MHz	69.4MHz	92.6MHz
Area	209251 μm^2	222572 μm^2	222978 μm^2
Power	8.3021mW	11.0574mW	14.8777mW

타이밍, 면적, 파워에 대한 결과를 정리하면 위와 같다.

SRCSA를 이용했을 때는 속도가 19.4% 빨라졌다. 이는 마지막 Vector merging adder의 딜레이가 22FA에서 7FA(5FA+2MUX)로 감소하여, 전체 회로의 CPD가 1AND+43FA에서 1AND+28FA로 크게 줄어들었기 때문이다. 파이프라인을 적용했을 때는 59.4% 빨라졌다. 이는 타이밍 리포트를 바탕으로 기존 크리티컬 패스의 중간 지점에 파이프라인 단계를 추가했기 때문에 가능했다. 하지만 파이프라인 레지스터의 clock-to-q delay, setup time으로 인해 이론적인 수치인 100%에는 미치지 못했다.

SRCSA는 carry를 미리 계산하고 선택하는 구조상 일반 RCA보다 더 많은 MUX와 추가적으로 20개의 FA가 필요하기 때문에 일반 RCA를 이용한 구조보다 13321 μm^2 만큼 면적이 늘어난 것을 볼 수 있다. 파이프라인을 적용한 구조에서는 파이프라인 레지스터와 플립플롭, 그리고 이를 연결해주기 위한 배선의 증가로 인해 면적이 13727 μm^2 증가했다. 다만 이 면적증가율은 그렇게 높지 않은데, 어떤 vector merge adder을 사용하더라도 이미 기본적으로 면적이 큰 21개의 22-bit adder를 사용해야 하기 때문이다.

SRCSA는 소모전력이 일반 RCA보다 33% 증가했다. 이것은 클럭 주파수 증가율보다 큰 수치인데, 소모 전력이 클럭 주파수에 선형적으로 비례하지 않을 수도 있고, 더 많은 가산기와 MUX가 사용됐기 때문에 예상보다 더 높은 증가율을 보였다고 생각해볼 수 있다. 파이프라인을 적용한 설계의 경우 79%의 전력 증가율을 보였다. 역시 클럭 주파수 증가율보다 더 높은 수치이다. 이는 클럭이 증가한 것과 더불어 파이프라인을 추가하기 위해 사용한 플립플롭과 레지스터가 전력을 소모하기 때문이다. SRCSA의 경우와 마찬가지로 소모 전력이 클럭 주파수에 선형적으로 비례하지 않는다면 그 사실 역시 원인이 될 수 있다.

속도만을 위해 최적화를 한다면 SRCSA보다 파이프라이닝을 적용하는 것이 훨씬 유리한 것으로 보인다. 둘 사이 면적도 크게 차이나지 않고, 소모 전력도 클럭 주파수를 감안하면 파이프라인을 적용한 구조가 훨씬 효율적이다. 하지만 면적이나 소모 전력을 줄여야

하는 상황에서는 일반 RCA를 이용한 구조가 더 나을 것이다.

c. Pipelining Strategies

최초에는 1AND+21FA:22FA의 비율로 경로를 나누어 파이프라인을 추가하면 되겠다고 생각을 하였으나, 합성 결과와 타이밍 리포트를 보고 그 접근이 잘못되었음을 알게 되었다. Critical path의 중간지점은 훨씬 앞쪽에 있어서 vector merge adder보다 훨씬 이전에 파이프라인을 추가해야 했다. 이는 앞서 있는 21개의 adder 사이의 배선으로 인한 딜레이가 1개의 22-bit RCA 내부의 배선으로 인한 딜레이보다 더 크기 때문이라고 추측해볼 수 있다.

최종적으로 파이프라인을 통해서 10.8ns라는 최소 클럭 주기를 얻고 나서, 그 클럭 조건을 다시 원래의 파이프라인 설계(21FA:22FA)에 적용해봤다. 그 결과 아래와 같은 타이밍 리포트를 얻을 수 있었는데 slack이 -2.25로 타이밍을 전혀 맞추지 못했다. 이를 통해 새로 설계한 파이프라인이 훨씬 좋은 설계임을 확인할 수 있다.

Startpoint: b_reg[1] (rising edge-triggered flip-flop clocked by clk)	0,00	8,65 f	0,00	11,98 f
Endpoint: csm_rca/mul_LSB_reg[19] (rising edge-triggered flip-flop clocked by clk)	0,00	8,65 f	0,00	11,98 f
Path Group: clk				
Path Type: max				
Point	Incr	Path		
clock clk (rise edge)	0,00	0,00		
clock network delay (ideal)	0,00	0,00		
b_reg[1]/CLK (dffs2)	0,00	0,00 r		
b_reg[1]/Q (dffs2)	0,37	0,37 f		
csm_rca/b[1] (CSM_RCA_pipelined)	0,00	0,37 f		
csm_rca/gen_and[1].u.and/b (and_gates_21)	0,00	0,37 f		
csm_rca/gen_and[1].u.and/U14/Q (and2s1)	0,41	0,78 f		
csm_rca/gen_and[1].u.and/out[19] (and_gates_21)	0,00	0,78 f		
csm_rca/u_adder0/u_add[19] (adder_22bit_wide_carry_0)	0,00	0,78 f		
csm_rca/u_adder0/fa_gen[19].u.f/a/a (fa_465)	0,00	0,78 f		
csm_rca/u_adder0/fa_gen[19].u.f/a/U1/Q (nm2s2)	0,44	1,22 r		
csm_rca/u_adder0/su[19] (adder_22bit_wide_carry_0)	0,00	1,22 r		
csm_rca/gen_adder[1].u_adder/b[18] (adder_22bit_wide_carry_20)	0,00	1,22 r		
csm_rca/gen_adder[1].u_adder/fa_gen[18].u.f/a/b (fa_444)	0,00	1,22 r		
csm_rca/gen_adder[1].u_adder/fa_gen[18].u.f/a/U1/Q (11s4)	0,07	1,29 f		
csm_rca/gen_adder[1].u_adder/fa_gen[18].u.f/a/U3/Q (nm3s2)	0,30	1,60 f		
csm_rca/gen_adder[1].u_adder/fa_gen[18].u.f/a/su (fa_444)	0,00	1,60 f		
csm_rca/gen_adder[1].u_adder/b[8] (adder_22bit_wide_carry_10)	0,00	6,96 r		
csm_rca/gen_adder[11].u_adder/fa_gen[8].u.f/a/b (fa_234)	0,00	6,96 r		
csm_rca/gen_adder[11].u_adder/fa_gen[8].u.f/a/U3/Q (vor2s3)	0,25	7,21 f		
csm_rca/gen_adder[11].u_adder/fa_gen[8].u.f/a/U4/Q (vor2s3)	0,34	7,55 r		
csm_rca/gen_adder[11].u_adder/fa_gen[8].u.f/a/su (fa_234)	0,00	7,55 r		
csm_rca/gen_adder[11].u_adder/su[8] (adder_22bit_wide_carry_10)	0,00	7,55 r		
csm_rca/gen_adder[12].u_adder/b[7] (adder_22bit_wide_carry_3)	0,00	7,55 r		
csm_rca/gen_adder[12].u_adder/fa_gen[7].u.f/a/b (fa_213)	0,00	7,55 r		
csm_rca/gen_adder[12].u_adder/fa_gen[7].u.f/a/U4/Q (vor2s3)	0,25	7,79 f		
csm_rca/gen_adder[12].u_adder/fa_gen[7].u.f/a/U6/Q (vor2s3)	0,36	8,15 r		
csm_rca/gen_adder[12].u_adder/fa_gen[7].u.f/a/su (fa_213)	0,00	8,15 r		
csm_rca/gen_adder[12].u_adder/su[7] (adder_22bit_wide_carry_9)	0,00	8,15 r		
csm_rca/gen_adder[13].u_adder/b[6] (adder_22bit_wide_carry_3)	0,00	8,15 r		
csm_rca/gen_adder[13].u_adder/fa_gen[6].u.f/a/b (fa_192)	0,00	8,15 r		
csm_rca/gen_adder[13].u_adder/fa_gen[6].u.f/a/U6/Q (vor2s2)	0,26	8,41 f		
csm_rca/gen_adder[13].u_adder/fa_gen[6].u.f/a/U7/Q (nm2s3)	0,13	8,54 r		
csm_rca/gen_adder[13].u_adder/fa_gen[6].u.f/a/U9/Q (nm2s3)	0,11	8,65 f		
csm_rca/gen_adder[14].u_adder/b[5] (adder_22bit_wide_carry_7)	0,00	8,65 f		
csm_rca/gen_adder[14].u_adder/fa_gen[5].u.f/a/b (fa_171)	0,00	8,65 f		
csm_rca/gen_adder[14].u_adder/fa_gen[5].u.f/a/U7/Q (vor2s3)	0,34	9,00 r		
csm_rca/gen_adder[14].u_adder/fa_gen[5].u.f/a/U6/Q (vor2s3)	0,25	9,25 f		
csm_rca/gen_adder[14].u_adder/fa_gen[5].u.f/a/su (fa_171)	0,00	9,25 f		
csm_rca/gen_adder[14].u_adder/su[5] (adder_22bit_wide_carry_7)	0,00	9,25 f		
csm_rca/gen_adder[15].u_adder/b[4] (adder_22bit_wide_carry_6)	0,00	9,25 f		
csm_rca/gen_adder[15].u_adder/fa_gen[4].u.f/a/b (fa_150)	0,00	9,25 f		
csm_rca/gen_adder[15].u_adder/fa_gen[4].u.f/a/U3/Q (vor2s3)	0,00	9,25 f		
csm_rca/gen_adder[15].u_adder/fa_gen[4].u.f/a/U1/Q (nm2s3)	0,35	9,60 r		
csm_rca/gen_adder[15].u_adder/fa_gen[4].u.f/a/U5/Q (nm2s3)	0,10	9,69 f		
csm_rca/gen_adder[15].u_adder/fa_gen[4].u.f/a/out (fa_150)	0,11	9,80 r		
csm_rca/gen_adder[15].u_adder/cout[4] (adder_22bit_wide_carry_6)	0,00	9,80 r		
csm_rca/gen_adder[16].u_adder/cin[4] (adder_22bit_wide_carry_5)	0,00	9,80 r		
csm_rca/gen_adder[16].u_adder/fa_gen[4].u.f/a/cin (fa_128)	0,00	9,80 r		
csm_rca/gen_adder[16].u_adder/fa_gen[4].u.f/a/U7/Q (vor3s2)	0,38	10,18 r		
csm_rca/gen_adder[16].u_adder/fa_gen[4].u.f/a/su (fa_128)	0,00	10,18 r		
csm_rca/gen_adder[16].u_adder/su[4] (adder_22bit_wide_carry_5)	0,00	10,18 r		
csm_rca/gen_adder[17].u_adder/b[3] (adder_22bit_wide_carry_4)	0,00	10,18 r		
csm_rca/gen_adder[17].u_adder/fa_gen[3].u.f/a/b (fa_107)	0,00	10,18 r		
csm_rca/gen_adder[17].u_adder/fa_gen[3].u.f/a/U2/Q (vor2s2)	0,26	10,44 f		
csm_rca/gen_adder[17].u_adder/fa_gen[3].u.f/a/U6/Q (vor2s3)	0,34	10,78 r		
csm_rca/gen_adder[17].u_adder/fa_gen[3].u.f/a/su (fa_107)	0,00	10,78 r		
csm_rca/gen_adder[17].u_adder/su[3] (adder_22bit_wide_carry_4)	0,00	10,78 r		
csm_rca/gen_adder[18].u_adder/b[2] (adder_22bit_wide_carry_3)	0,00	10,78 r		
csm_rca/gen_adder[18].u_adder/fa_gen[2].u.f/a/b (fa_86)	0,00	10,78 r		
csm_rca/gen_adder[18].u_adder/fa_gen[2].u.f/a/U4/Q (vor2s3)	0,28	11,07 f		
csm_rca/gen_adder[18].u_adder/fa_gen[2].u.f/a/U3/Q (vor2s3)	0,35	11,41 f		
csm_rca/gen_adder[18].u_adder/fa_gen[2].u.f/a/su (fa_86)	0,00	11,41 f		
csm_rca/gen_adder[18].u_adder/su[2] (adder_22bit_wide_carry_3)	0,00	11,41 f		
csm_rca/gen_adder[19].u_adder/b[1] (adder_22bit_wide_carry_2)	0,00	11,41 f		
csm_rca/gen_adder[19].u_adder/fa_gen[1].u.f/a/b (fa_65)	0,00	11,41 f		
csm_rca/gen_adder[19].u_adder/fa_gen[1].u.f/a/U3/Q (vor2s3)	0,33	11,74 r		
csm_rca/gen_adder[19].u_adder/fa_gen[1].u.f/a/U4/Q (vor2s3)	0,25	11,90 f		
csm_rca/gen_adder[19].u_adder/fa_gen[1].u.f/a/su (fa_65)	0,00			